

```
import pandas as pd

df = pd.read_csv("marketing_sales_data.csv")

df.head()
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 572 entries, 0 to 571
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0    TV              572 non-null    object
1    Radio           572 non-null    float64
2    Social Media    572 non-null    float64
3    Influencer      572 non-null    object
4    Sales           572 non-null    float64
dtypes: float64(3), object(2)
memory usage: 22.5+ KB
```

	Radio	Social Media	Sales
count	572.000000	572.000000	572.000000
mean	17.520616	3.333803	189.296908
std	9.290933	2.238378	89.871581
min	0.109106	0.000031	33.509810
25%	10.699556	1.585549	118.718722
50%	17.149517	3.150111	184.005362
75%	24.606396	4.730408	264.500118
max	42.271579	11.403625	357.788195

```
df.isnull().sum()

df["Sales"] = df["Sales"].fillna(df["Sales"].mean())
```

```
df = df.dropna()
```

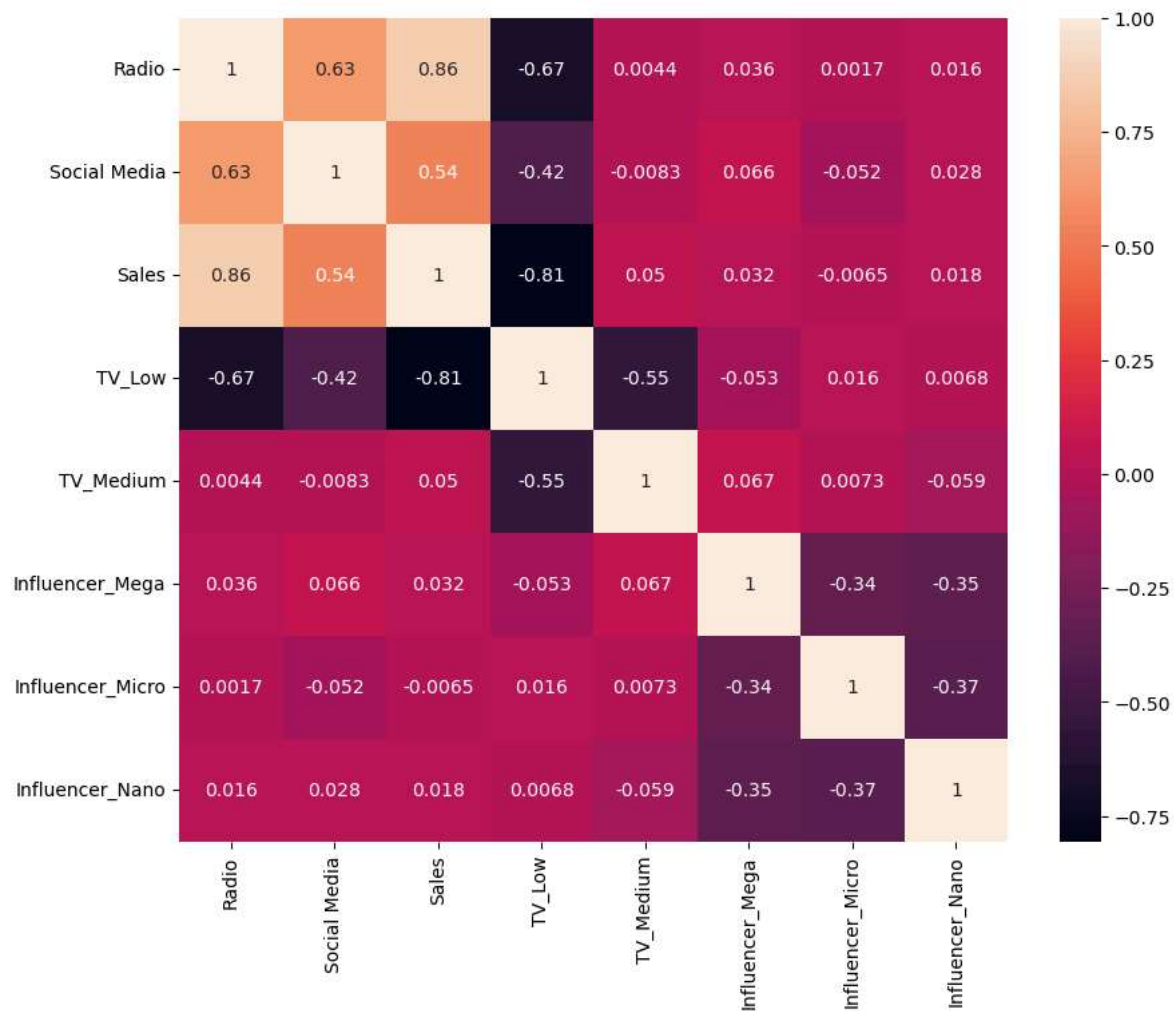
```
df_encoded = pd.get_dummies(
    df,
    columns=["TV", "Influencer"],
    drop_first=True
)
```

```
import seaborn as sns
import matplotlib.pyplot as plt

corr = df_encoded.corr(numeric_only=True)

plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True)

plt.show()
```



```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import statsmodels.api as sm
```

```
# Create dummy variables
df_encoded = pd.get_dummies(
    df,
    columns=["TV", "Influencer"],
    drop_first=True
)

# Convert all columns to numeric
df_encoded = df_encoded.astype(float)

# Define predictors
X = df_encoded.drop("Sales", axis=1)

# Add constant
X = sm.add_constant(X)

# Calculate VIF
vif = pd.DataFrame()
vif["Feature"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]

print(vif)
```

	Feature	VIF
0	const	31.649565
1	Radio	3.479641
2	Social Media	1.669098
3	TV_Low	4.076384

```

4      TV_Medium    2.233350
5  Influencer_Mega    1.593889
6  Influencer_Micro    1.618434
7  Influencer_Nano    1.627430

```

```

# Target variable
y = df_encoded["Sales"]

# Fit OLS model
model = sm.OLS(y, X).fit()

# Display results
print(model.summary())

```

```

                        OLS Regression Results
=====
Dep. Variable:          Sales    R-squared:                0.904
Model:                  OLS      Adj. R-squared:            0.903
Method:                 Least Squares    F-statistic:         760.4
Date:                  Wed, 17 Jun 2026    Prob (F-statistic):    1.82e-282
Time:                  03:47:55    Log-Likelihood:        -2713.4
No. Observations:      572    AIC:                  5443.
Df Residuals:          564    BIC:                  5478.
Df Model:              7
Covariance Type:       nonrobust
=====
                        coef    std err          t      P>|t|      [0.025     0.975]
-----
const                217.4784     6.584     33.031     0.000     204.546     230.411
Radio                 2.9735     0.235     12.644     0.000       2.512       3.435
Social Media        -0.1391     0.676     -0.206     0.837      -1.467       1.189
TV_Low             -154.5736     4.949    -31.231     0.000    -164.295    -144.852
TV_Medium          -75.5947     3.647    -20.726     0.000     -82.759     -68.431
Influencer_Mega     2.4948     3.462     0.721     0.471      -4.305       9.295
Influencer_Micro     2.9391     3.378     0.870     0.385      -3.695       9.574
Influencer_Nano      0.8015     3.346     0.240     0.811      -5.770       7.373
=====
Omnibus:              58.711    Durbin-Watson:         1.874
Prob(Omnibus):        0.000    Jarque-Bera (JB):      17.802
Skew:                 0.057    Prob(JB):              0.000136
Kurtosis:             2.143    Cond. No.              147.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

print("Adjusted R-squared:")
print(model.rsquared_adj)

```

```

Adjusted R-squared:
0.9030011006551627

```

```

predictions = model.predict(X)

residuals = y - predictions

```

```

import seaborn as sns
import matplotlib.pyplot as plt

sns.scatterplot(
    x=predictions,
    y=residuals
)

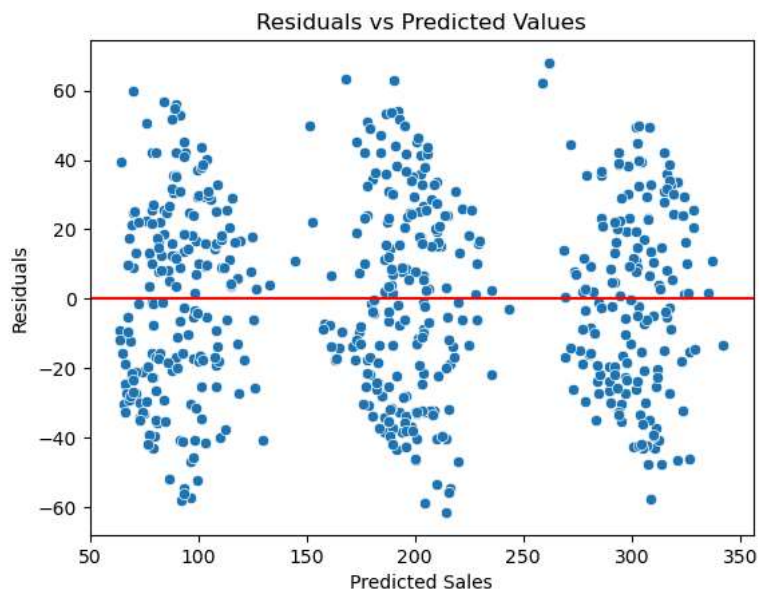
plt.axhline(y=0, color='red')

plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")

plt.title("Residuals vs Predicted Values")

plt.show()

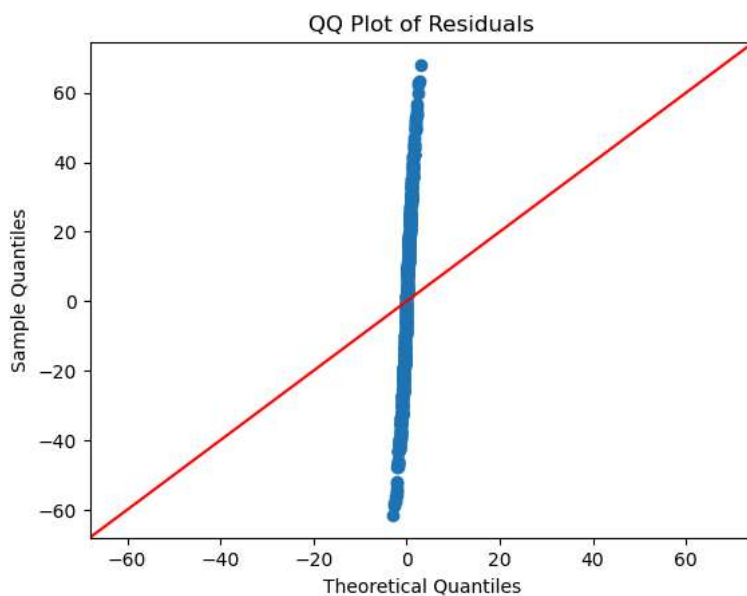
```



```
sm.qqplot(
    residuals,
    line='45'
)

plt.title("QQ Plot of Residuals")

plt.show()
```



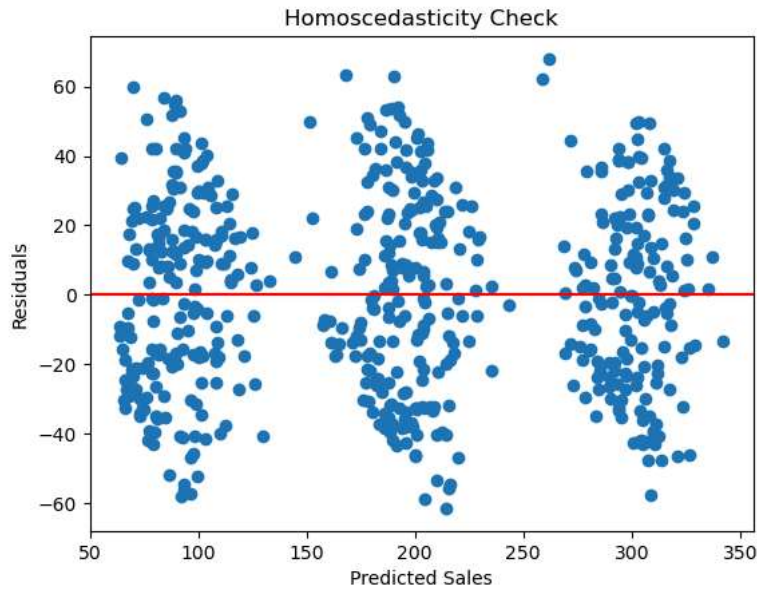
```
plt.scatter(
    predictions,
    residuals
)

plt.axhline(
    y=0,
    color='red'
)

plt.xlabel("Predicted Sales")
plt.ylabel("Residuals")

plt.title("Homoscedasticity Check")

plt.show()
```



```
print(model.summary())
```

OLS Regression Results						
=====						
Dep. Variable:	Sales	R-squared:	0.904			
Model:	OLS	Adj. R-squared:	0.903			
Method:	Least Squares	F-statistic:	760.4			
Date:	Wed, 17 Jun 2026	Prob (F-statistic):	1.82e-282			
Time:	03:48:06	Log-Likelihood:	-2713.4			
No. Observations:	572	AIC:	5443.			
Df Residuals:	564	BIC:	5478.			
Df Model:	7					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

const	217.4784	6.584	33.031	0.000	204.546	230.411
Radio	2.9735	0.235	12.644	0.000	2.512	3.435
Social Media	-0.1391	0.676	-0.206	0.837	-1.467	1.189
TV_Low	-154.5736	4.949	-31.231	0.000	-164.295	-144.852
TV_Medium	-75.5947	3.647	-20.726	0.000	-82.759	-68.431
Influencer_Mega	2.4948	3.462	0.721	0.471	-4.305	9.295
Influencer_Micro	2.9391	3.378	0.870	0.385	-3.695	9.574
Influencer_Nano	0.8015	3.346	0.240	0.811	-5.770	7.373
=====						
Omnibus:	58.711	Durbin-Watson:		1.874		
Prob(Omnibus):	0.000	Jarque-Bera (JB):		17.802		
Skew:	0.057	Prob(JB):		0.000136		
Kurtosis:	2.143	Cond. No.		147.		
=====						

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

#Recommendation

#Based on the multiple regression analysis, marketing channels with statistically significant coefficients should receive prior allocation. These channels demonstrate measurable impact on Sales while controlling for the influence of other marketing activ
#The model provides evidence-based guidance for optimizing marketing ROI and improving future budget decisions.

```
print(model.summary())
```

OLS Regression Results			
=====			
Dep. Variable:	Sales	R-squared:	0.904
Model:	OLS	Adj. R-squared:	0.903
Method:	Least Squares	F-statistic:	760.4
Date:	Wed, 17 Jun 2026	Prob (F-statistic):	1.82e-282
Time:	03:48:10	Log-Likelihood:	-2713.4
No. Observations:	572	AIC:	5443.
Df Residuals:	564	BIC:	5478.
Df Model:	7		
Covariance Type:	nonrobust		
=====			

	coef	std err	t	P> t	[0.025	0.975]
const	217.4784	6.584	33.031	0.000	204.546	230.411
Radio	2.9735	0.235	12.644	0.000	2.512	3.435
Social Media	-0.1391	0.676	-0.206	0.837	-1.467	1.189
TV_Low	-154.5736	4.949	-31.231	0.000	-164.295	-144.852
TV_Medium	-75.5947	3.647	-20.726	0.000	-82.759	-68.431
Influencer_Mega	2.4948	3.462	0.721	0.471	-4.305	9.295
Influencer_Micro	2.9391	3.378	0.870	0.385	-3.695	9.574
Influencer_Nano	0.8015	3.346	0.240	0.811	-5.770	7.373

Omnibus:	58.711	Durbin-Watson:	1.874
Prob(Omnibus):	0.000	Jarque-Bera (JB):	17.802
Skew:	0.057	Prob(JB):	0.000136
Kurtosis:	2.143	Cond. No.	147.

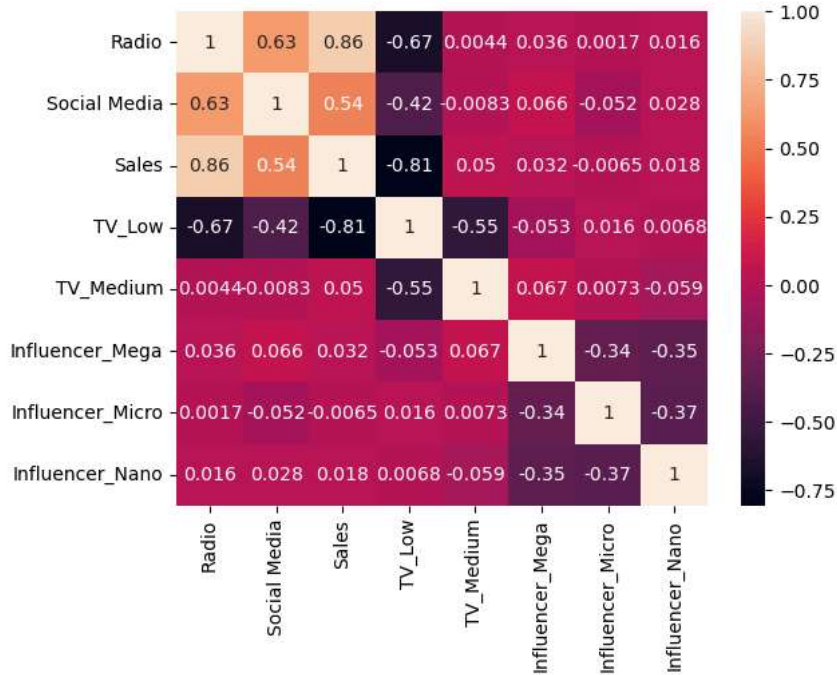
Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
corr = df_encoded.corr(numeric_only=True)
```

```
sns.heatmap(
    corr,
    annot=True
)
```

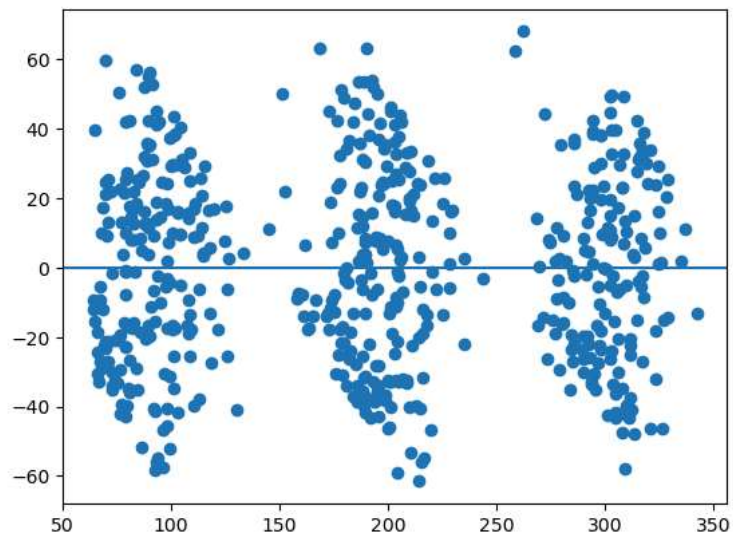
```
plt.show()
```



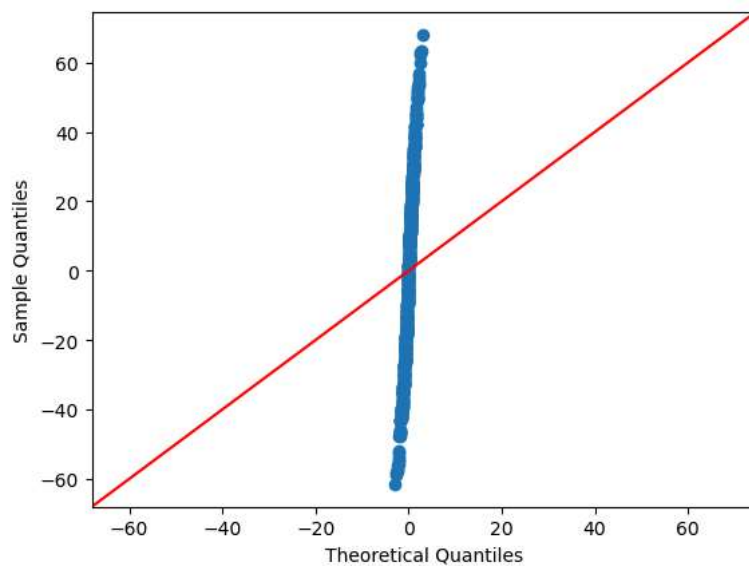
```
print(vif)
```

	Feature	VIF
0	const	31.649565
1	Radio	3.479641
2	Social Media	1.669098
3	TV_Low	4.076384
4	TV_Medium	2.233350
5	Influencer_Mega	1.593889
6	Influencer_Micro	1.618434
7	Influencer_Nano	1.627430

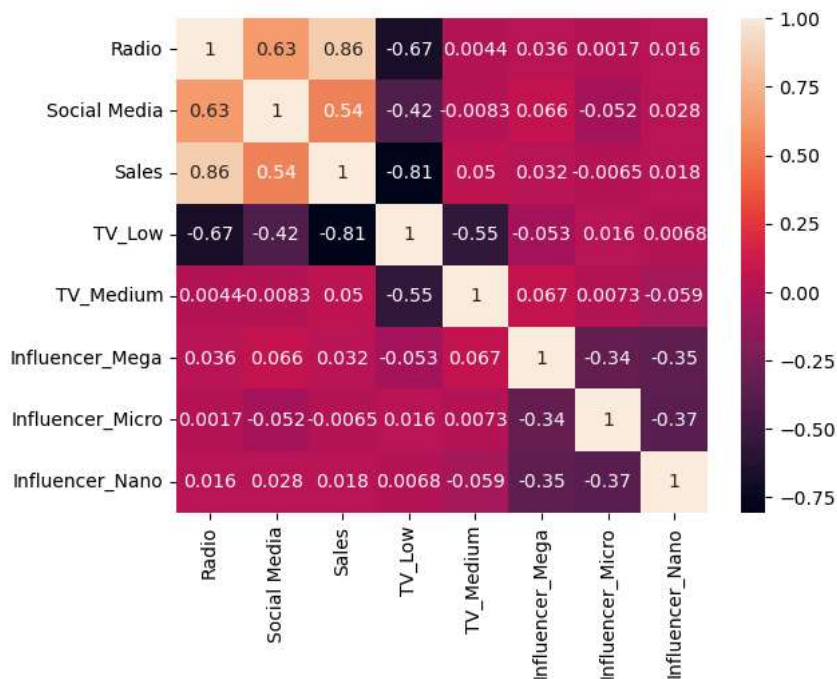
```
plt.scatter(predictions,residuals)
plt.axhline(0)
plt.show()
```



```
sm.qqplot(  
    residuals,  
    line='45'  
)  
  
plt.show()
```



```
corr = df_encoded.corr(numeric_only=True)  
  
sns.heatmap(corr, annot=True)  
plt.show()
```



```
print(vif)
```

	Feature	VIF
0	const	31.649565
1	Radio	3.479641
2	Social Media	1.669098
3	TV_Low	4.076384
4	TV_Medium	2.233350
5	Influencer_Mega	1.593889
6	Influencer_Micro	1.618434
7	Influencer_Nano	1.627430

```
model = sm.OLS(y, X).fit()
```

```
print(model.summary())
```

```

=====
                        OLS Regression Results
=====
Dep. Variable:          Sales      R-squared:                0.904
Model:                  OLS       Adj. R-squared:           0.903
Method:                 Least Squares   F-statistic:             760.4
Date:                  Wed, 17 Jun 2026   Prob (F-statistic):      1.82e-282
Time:                  03:50:17         Log-Likelihood:          -2713.4
No. Observations:      572            AIC:                    5443.
Df Residuals:          564            BIC:                    5478.
Df Model:              7
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	217.4784	6.584	33.031	0.000	204.546	230.411
Radio	2.9735	0.235	12.644	0.000	2.512	3.435
Social Media	-0.1391	0.676	-0.206	0.837	-1.467	1.189
TV_Low	-154.5736	4.949	-31.231	0.000	-164.295	-144.852
TV_Medium	-75.5947	3.647	-20.726	0.000	-82.759	-68.431
Influencer_Mega	2.4948	3.462	0.721	0.471	-4.305	9.295
Influencer_Micro	2.9391	3.378	0.870	0.385	-3.695	9.574
Influencer_Nano	0.8015	3.346	0.240	0.811	-5.770	7.373

```

=====
Omnibus:                 58.711   Durbin-Watson:           1.874
Prob(Omnibus):           0.000   Jarque-Bera (JB):         17.802
Skew:                   0.057   Prob(JB):                 0.000136
Kurtosis:               2.143   Cond. No.                 147.
=====

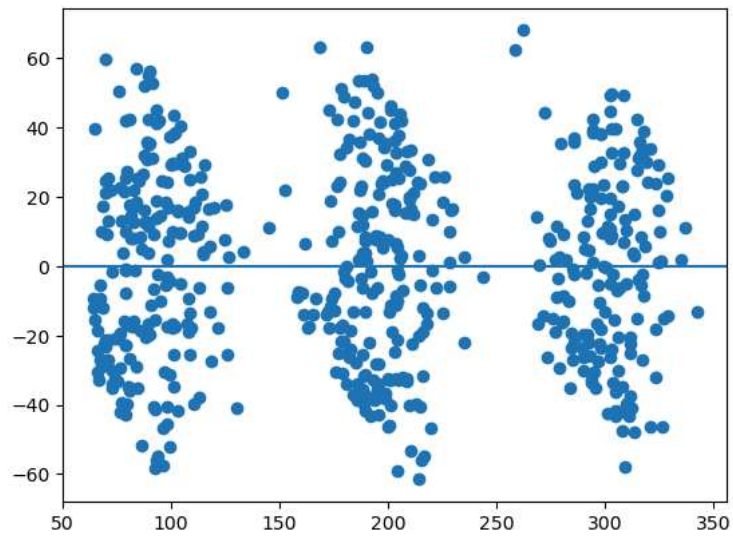
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
plt.scatter(predictions, residuals)
plt.axhline(0)
```

```
plt.show()
```



```
sm.qqplot(residuals, line='45')  
plt.show()
```